DSA Streak: Daily LeetCode in Java

2026-11-13 | 5 problems

#	Problem	Difficulty	Topic
1	Daily Temperatures	Medium	Array, Stack
2	Best Time to Buy and Sell Stock	Easy	Array, Dynamic Programming
3	Product of Array Except Self	Medium	Array, Prefix Sum
4	Koko Eating Bananas	Medium	Array, Binary Search
5	N-Queens	Hard	Array, Backtracking

1. Daily Temperatures

Medium | #739 | Array, Stack, Monotonic Stack
https://leetcode.com/problems/daily-temperatures/

Problem

You get the temperature for each day. For every day, find how many days you must wait until a strictly warmer day. If no warmer day comes later, the answer for that day is 0.

Examples

Example 1	
Input	`temperatures = [73,74,75,71,69,72,76,73]`
Output	`[1,1,4,2,1,1,0,0]`
Why	Day 0 (73) is beaten the next day by 74. Day 2 (75) waits until day 6 (76), so 4 days. Days 6 and 7 never see a warmer day.

Example 2	
Input	`temperatures = [30,40,50,60]`
Output	`[1,1,1,0]`
Why	Every day is beaten by the next day except the last one.

Example 3	
Input	`temperatures = [30,60,90]`
Output	`[1,1,0]`
Why	Strictly increasing, so each day waits 1 day, last day gets 0.

Intuition

Keep a stack of days that are still waiting for a warmer day. Their temperatures are decreasing from bottom to top, because a warmer day would already have resolved the colder ones. When today is warmer than the day on top, that day has found its answer, so pop it and record the gap. Every index is pushed and popped at most once.

Approach

1. Create ans of size n filled with 0 and an empty stack st of indices.
2. For each day i, while st is not empty and temperatures[st.peek()] < temperatures[i], pop j and set ans[j] = i - j.
3. Push i onto st since it is now waiting for its own warmer day.
4. Indices left in st never found a warmer day, so they stay 0. Return ans.

Dry run on Example 1

- temperatures = [73,74,75,71,69,72,76,73], st = []
- i=0 (73): st empty, push 0. st = [0]
- i=1 (74): 73 < 74, pop j=0, ans[0]=1. push 1. st = [1]
- i=2 (75): 74 < 75, pop j=1, ans[1]=1. push 2. st = [2]
- i=3 (71): 75 not < 71, push 3. st = [2,3]
- i=4 (69): 71 not < 69, push 4. st = [2,3,4]
- i=5 (72): pop j=4, ans[4]=1; pop j=3, ans[3]=2; 75 not < 72, push 5. st = [2,5]
- i=6 (76): pop j=5, ans[5]=1; pop j=2, ans[2]=4; push 6. st = [6]
- i=7 (73): 76 not < 73, push 7. st = [6,7]
- Loop ends, ans = [1,1,4,2,1,1,0,0]

Java solution

```
import java.util.*;

class Solution {
    public int[] dailyTemperatures(int[] temperatures) {
        int n = temperatures.length;
        int[] ans = new int[n];
        Deque<Integer> st = new ArrayDeque<>();
        for (int i = 0; i < n; i++) {
            while (!st.isEmpty() && temperatures[st.peek()] < temperatures[i]) {
                int j = st.pop();
                ans[j] = i - j;
            }
            st.push(i);
        }
        return ans;
    }
}
```

Complexity

Time: O(n), each index is pushed and popped once Space: O(n) for the stack

Edge cases handled

- Single day: answer is [0]
- Equal temperatures do not count as warmer, so all 0
- Strictly decreasing input: stack grows to n and every answer is 0
- n = 100000 with one hot last day: linear, no timeout

Pattern to remember

Next greater element to the right means a monotonic decreasing stack of indices; pop when the current value beats the top.

2. Best Time to Buy and Sell Stock

Easy | #121 | Array, Dynamic Programming
https://leetcode.com/problems/best-time-to-buy-and-sell-stock/

Problem

prices[i] is the stock price on day i. Buy once and sell once on a later day to get the maximum profit. If no profit is possible, return 0.

Examples

Example 1	
Input	`prices = [7,1,5,3,6,4]`
Output	`5`
Why	Buy on day 2 (price 1) and sell on day 5 (price 6), profit 6 - 1 = 5. Buying at 1 and selling at 7 is not allowed because you must buy before you sell.

Example 2	
Input	`prices = [7,6,4,3,1]`
Output	`0`
Why	Prices only fall, so no transaction gives a profit and the answer is 0.

Intuition

For any sell day, the best buy day is the cheapest day before it. So walk left to right, remember the lowest price seen so far in min, and check what selling today would earn.

Approach

1. Set min = Integer.MAX_VALUE and ans = 0.
2. For each price p: if p < min, update min (a better day to buy).
3. Otherwise compute p - min and keep the larger of that and ans.
4. Return ans.

Dry run on Example 1

- prices = [7,1,5,3,6,4], min = MAX, ans = 0
- p=7: 7 < min, min = 7
- p=1: 1 < 7, min = 1
- p=5: ans = max(0, 5-1) = 4
- p=3: ans = max(4, 3-1) = 4
- p=6: ans = max(4, 6-1) = 5
- p=4: ans = max(5, 4-1) = 5
- Return 5

Java solution

```
class Solution {
    public int maxProfit(int[] prices) {
        int min = Integer.MAX_VALUE, ans = 0;
        for (int p : prices) {
            if (p < min) min = p;
            else ans = Math.max(ans, p - min);
        }
        return ans;
    }
}
```

Complexity

Time: O(n) Space: O(1)

Edge cases handled

- Single day: no sell possible, return 0
- All prices equal: profit 0
- Strictly decreasing prices: min keeps dropping, ans stays 0
- Best buy is not the global minimum if the minimum comes at the end, e.g. [2,4,1] gives 2

Pattern to remember

Track the best value seen so far on the left (running min) and combine it with the current element; this is the base of many one-pass array problems.

3. Product of Array Except Self

Medium | #238 | Array, Prefix Sum
https://leetcode.com/problems/product-of-array-except-self/

Problem

Return an array where answer[i] is the product of every element of nums except nums[i]. Division is not allowed and it must run in O(n).

Examples

Example 1	
Input	`nums = [1,2,3,4]`
Output	`[24,12,8,6]`
Why	24 = 2*3*4, 12 = 1*3*4, 8 = 1*2*4, 6 = 1*2*3.

Example 2	
Input	`nums = [-1,1,0,-3,3]`
Output	`[0,0,9,0,0]`
Why	Every product that includes the 0 is 0. Only index 2 skips the 0, giving -1*1*-3*3 = 9.

Intuition

The product of everything except i equals (product of everything to the left of i) times (product of everything to the right of i). Store the left products in res in one pass, then sweep from the right with a running product r and multiply it in. No division, so zeros are handled naturally.

Approach

1. Set res[0] = 1. For i from 1 to n-1, res[i] = res[i-1] * nums[i-1] (left product).
2. Set r = 1 (product of elements to the right).
3. For i from n-1 down to 0: res[i] *= r, then r *= nums[i].
4. Return res.

Dry run on Example 1

- nums = [1,2,3,4]
- Left pass: res = [1, 1, 1*2=2, 2*3=6]
- Right pass, r = 1
- i=3: res[3] = 6*1 = 6, r = 1*4 = 4
- i=2: res[2] = 2*4 = 8, r = 4*3 = 12
- i=1: res[1] = 1*12 = 12, r = 12*2 = 24
- i=0: res[0] = 1*24 = 24, r = 24
- Return [24,12,8,6]

Java solution

```
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] res = new int[n];
        res[0] = 1;
        for (int i = 1; i < n; i++) res[i] = res[i - 1] * nums[i - 1];
        int r = 1;
        for (int i = n - 1; i >= 0; i--) {
            res[i] *= r;
            r *= nums[i];
        }
        return res;
```

```
    }
}
```

Complexity

Time: O(n) Space: O(1) extra (the output array does not count)

Edge cases handled

- Minimum size n = 2: each answer is the other element
- One zero: only the zero's index gets a non zero product
- Two or more zeros: every answer is 0
- Negative numbers: signs work out automatically through multiplication

Pattern to remember

When an answer depends on 'everything except i', split it into a prefix part and a suffix part and combine them in two passes.

4. Koko Eating Bananas

Medium | #875 | Array, Binary Search
https://leetcode.com/problems/koko-eating-bananas/

Problem

There are piles of bananas and h hours. Each hour Koko picks one pile and eats up to k bananas from it (if the pile has fewer, she finishes it and waits for the rest of the hour). Find the smallest integer speed k that lets her finish all piles within h hours.

Examples

Example 1	
Input	`piles = [3,6,7,11], h = 8`
Output	`4`
Why	At k = 4 she needs 1+2+2+3 = 8 hours. At k = 3 she would need 1+2+3+4 = 10 hours, too slow.

Example 2	
Input	`piles = [30,11,23,4,20], h = 5`
Output	`30`
Why	Only 5 hours for 5 piles, so each pile must be finished in one hour, so k must be the largest pile, 30.

Example 3	
Input	`piles = [30,11,23,4,20], h = 6`
Output	`23`
Why	With one spare hour, k = 23 works: 2+1+1+1+1 = 6 hours.

Intuition

Hours needed only go down as k goes up, so the answer space is sorted: too slow, too slow, ..., fast enough, fast enough. Binary search for the first k that is fast enough. k is between 1 and the largest pile (eating faster than the biggest pile never helps).

Approach

1. l = 1, r = max pile.
2. While l < r: mid = l + (r - l) / 2.
3. hrs = sum of ceil(p / mid) over all piles, computed as (p + mid - 1) / mid and stored in a long.
4. If hrs <= h, mid works, so try smaller: r = mid. Else l = mid + 1.
5. When l == r, return l.

Dry run on Example 1

- piles = [3,6,7,11], h = 8, l = 1, r = 11
- mid = 6: hrs = 1+1+2+2 = 6 <= 8, r = 6
- mid = 3: hrs = 1+2+3+4 = 10 > 8, l = 4
- mid = 5: hrs = 1+2+2+3 = 8 <= 8, r = 5
- mid = 4: hrs = 1+2+2+3 = 8 <= 8, r = 4
- l == r == 4, return 4

Java solution

```
class Solution {
```

```
    public int minEatingSpeed(int[] piles, int h) {
        int l = 1, r = 0;
        for (int p : piles) r = Math.max(r, p);
        while (l < r) {
            int mid = l + (r - l) / 2;
            long hrs = 0;
            for (int p : piles) hrs += (p + mid - 1) / mid;
            if (hrs <= h) r = mid;
            else l = mid + 1;
        }
        return l;
    }
}
```

Complexity

Time: O(n log M) where M is the largest pile Space: O(1)

Edge cases handled

- h equals number of piles: answer is the max pile
- h very large (1e9): answer can drop to 1
- Piles up to 1e9: p + mid - 1 is at most 1999999999, which still fits in an int, and hrs is a long so adding 10^4 large counts cannot overflow
- Single pile: answer is ceil(pile / h)

Pattern to remember

Binary search on the answer: if a yes/no check is monotonic in some value, search the smallest value where the check says yes.

5. N-Queens

Hard | #51 | Array, Backtracking
https://leetcode.com/problems/n-queens/

Problem

Place n queens on an n x n chessboard so that no two queens attack each other (same row, column or diagonal). Return every distinct board, written as strings where 'Q' is a queen and '.' is empty. Any order is fine.

Examples

Example 1	
Input	`n = 4`
Output	`[[".Q..","...Q","Q...","..Q."],["..Q.","Q...","...Q",".Q.."]]`
Why	There are exactly two distinct solutions to the 4-queens puzzle.

Example 2	
Input	`n = 1`
Output	`[["Q"]]`
Why	One queen on a 1 x 1 board.

Intuition

Every row must hold exactly one queen, so fill row by row and only choose the column. A square (r, c) is attacked if its column, its main diagonal (r + c is constant on it) or its anti diagonal (r - c is constant, shifted by n to stay non negative) is already used. Three boolean arrays make each check O(1), and we undo the choice after recursing (backtracking).

Approach

1. Fill board with '.', create col[n], d1[2n], d2[2n].
2. solve(r): if r == n, copy the board rows into a list and add it to res.
3. Otherwise for each column c, skip if col[c], d1[r + c] or d2[r - c + n] is true.
4. Mark all three, put 'Q' at board[r][c], call solve(r + 1).
5. Undo: put '.' back and unmark all three, then try the next column.

Dry run on Example 1

- n = 4, start solve(0)
- r=0, c=0: place. r=1: c=0 col used, c=1 d2[4] used, c=2 ok place
- r=2: c=0 col, c=1 d1[3] used, c=2 col, c=3 d2[3] used. Dead end, undo (1,2)
- r=1, c=3: place. r=2: c=0 col, c=1 ok place. r=3: every column blocked. Undo back up to r=0
- r=0, c=1: place. r=1: c=0 d1[1] used, c=1 col, c=2 d2[3] used, c=3 ok place
- r=2: c=0 ok place (d1[2], d2[6] free). r=3: c=2 ok (d1[5], d2[5] free)
- r == 4, add [".Q..","...Q","Q...","..Q."] to res
- Continue: r=0, c=2 gives the mirror board ["..Q.","Q...","...Q",".Q.."]; c=3 finds nothing. res has 2 boards

Java solution

```
import java.util.*;

class Solution {
    List<List<String>> res = new ArrayList<>();
    char[][] board;
    boolean[] col, d1, d2;

    public List<List<String>> solveNQueens(int n) {
        board = new char[n][n];
```

```
        for (char[] row : board) Arrays.fill(row, '.');
        col = new boolean[n];
        d1 = new boolean[2 * n];
        d2 = new boolean[2 * n];
        solve(0, n);
        return res;
    }

    void solve(int r, int n) {
        if (r == n) {
            List<String> cur = new ArrayList<>();
            for (char[] row : board) cur.add(new String(row));
            res.add(cur);
            return;
        }
        for (int c = 0; c < n; c++) {
            if (col[c] || d1[r + c] || d2[r - c + n]) continue;
            col[c] = d1[r + c] = d2[r - c + n] = true;
            board[r][c] = 'Q';
            solve(r + 1, n);
            board[r][c] = '.';
            col[c] = d1[r + c] = d2[r - c + n] = false;
        }
    }
}
```

Complexity

Time: O(n!) roughly, since each row tries fewer columns than the one before Space: O(n^2) for the board plus O(n) recursion depth (output not counted)

Edge cases handled

- n = 1: single board ["Q"]
- n = 2 and n = 3: no solutions, return an empty list
- n = 9 (max): 352 boards, still fast with O(1) checks
- Remember to copy the board when saving, otherwise later backtracking overwrites it

Pattern to remember

Backtracking with constraint arrays: place, mark, recurse, unmark. Diagonals are r + c and r - c.